

IIFE(Immediate invoke function expressions)

✓ **Is defined and executed immediately after it is created.**

“An IIFE is a function expression that executes immediately after creation, used to create an isolated scope and avoid global namespace pollution.”

Why IIFE Exists

In JavaScript, variables declared globally can **pollute the global scope** and cause conflicts.

✗ Problem Without IIFE

```
var name = "Prajwal";
```

```
function init() {  
  var age = 22;  
}
```

```
init();
```

If multiple scripts define name, they overwrite each other.

✓ IIFE Solves This

An IIFE creates a **private scope**.

```
(function () {  
  var name = "Prajwal";  
  console.log(name);  
})();
```

- ✓ Runs immediately
- ✓ Variables stay private
- ✓ Nothing leaks to global scope

2. Basic Syntax

```
(function () {  
  // code runs instantly  
})();
```

! Why Wrap Function in () ?

JavaScript treats this as invalid:

```
function () {} // ✗ error (needs a name)
```

But wrapping makes it an **expression**:

```
(function () {}) // ✓ valid expression
```

Then we invoke it:

```
(function () {}());
```

Named IIFE (Rare but Asked in Interviews)

```
(function init() {  
  console.log("Runs immediately");  
})();
```

The name `init` is **only accessible inside**, not outside.

4. IIFE With Parameters

You can pass values just like a normal function.

```
(function (name) {  
  console.log("Hello " + name);  
})("Prajwal");  
  
// Hello Prajwal
```

IIFE as a Module Pattern (Classic Interview Topic)

Before ES6 import/export, developers built modules like this:

```
const UserModule = (function () {  
  let users = [];  
  
  function addUser(name) {  
    users.push(name);  
  }  
  
  function listUsers() {  
    console.log(users);  
  }  
  
  return {  
    addUser,  
    listUsers  
  };  
})();
```

Usage:

```
UserModule.addUser("Prajwal");  
UserModule.listUsers();
```

But users cannot be accessed directly → **true data hiding**.

8. Async IIFE (Used Today!)

Very common in Node.js when using await at top level.

```
(async function () {  
  const data = await fetchData();  
  console.log(data);  
})();
```

Because earlier JS didn't allow top-level await.

Common Interview Traps

✖ Missing Semicolon Issue

If previous code doesn't end with `;`, IIFE may break.

```
const a = 10
(function () {}());
```

JS thinks you're calling `10(...)` → 🌟 error.

Thinking IIFE Runs Only Once

Wrong. It runs **once per definition**.

```
(function () {
  console.log("Runs");
})();
```

```
(function () {
  console.log("Runs again");
})();
```

2. Why do we use IIFE in JavaScript?

Key Reasons:

- Avoid **global scope pollution**
- Create **private variables (encapsulation)**
- Execute setup code immediately
- Used in **module pattern (before ES6 modules)**
- Prevent variable conflicts in large apps

14. How Many Times Does an IIFE Run?

👉 Exactly **once**, at the moment it is defined.

It does NOT stay in memory like a normal function unless it returns something.

15. Is IIFE Still Needed After ES6 Modules?

Great senior-level question.

Modern replacement:

- import/export
- block scope (let, const)
- native modules

But IIFE is still useful for:

- sandbox execution
- one-time initialization
- avoiding leaks in legacy systems